Cache-Simulation und Analyse

PROJEKT A11

GROUP 221 : ADRIAN, SAMHITHA, YAZEED

Contents

1. Introduction to Caches

- Caches are a kind of data memory, that temporarily stores data and helps remove the burden of the main memory
- It was developed in answer to Moore's Law about growing processor sizes and growing memory access times
- Usually range from 300Kb - 16MB in size
-
-

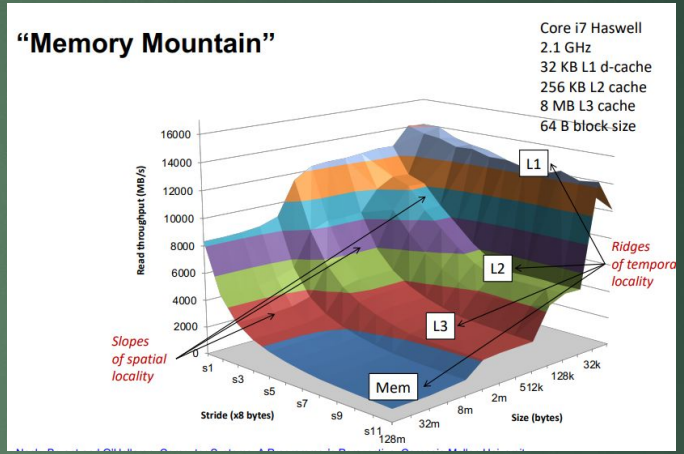
1. Introduction to Caches

Terminology :

1. Cache-latency : The time (in cycles) between a request to the cache and the actual data access
2. Memory-latency : The time between a data request and the arrival of the data from the cache
3. Cache line size : The amount of data transferred between cache and main memory
4. Cache Miss : When data is either not found for a read request, or when the cache line is empty for a write request
    a. Three types : cold/compulsory miss (first ever access on a line), capacity miss, conflict miss
5. Cache Hit : the opposite of a cache miss

2. Structure of Caches

- Types of caches :
  - direct mapped
  - n-way associative
  - fully associative
  - Other lesser used structures (
- Cache hierarchy
  - L1 > L2 > L3 …
- Different placement/replacement strategies for easier design
  - Least Recently Used, Least Frequently Used
- Renewal strategies : Write/No-write allocation, Write-around



3. Structure of a cache address

- The tag determines which data is stored
- sort of “bookmark” in the cache itself.
- The index determines in which cache line the data is stored
- The offset determines which byte of the cache line is accessed

- We calculate it this way :
  - Cache sets = cache lines / associativity (1 if it’s direct mapped)
  - log_2(cache line size) amount of bits we need for the offset. Remove that amount of bits from the end of the address
  - log_2 (cache sets) for the index
  - The bits that are left are the tag

4. Saving Data in a cache

- Insert the data at [offset] byte of the [index] line of the cache (direct mapped) or [index] set of the cache (n-way)
- If it's not empty, return a cache hit, if it is empty, return a miss
- If it's a hit or a miss on an associative cache and the cache line is full, replace the cache line as per replacement policy
- If it's direct mapped, the cache is full and an error is returned

5. Adrian's work

- Implementation of the complete C program
  - Worked with getopt_long
  - created a .csv parser
  - Created a help manual and a command line parser
- Research about sizes and latencies of modern caches
- Research about SelectionSort
- Creation of the example csv files

Sizes and latencies of modern caches

Name	Type	L1- Size	L2 - Size	L3 - Size
AMD Ryzen 5 PRO 7640HS	Notebook Processor	384 KB	6MB	16MB
AMD Ryzen 7 3700	Regular Laptop Processor	384 KB	2 MB	4MB
AMD Ryzen 7 5700G	Desktop-APU	512 KB	4MB	16MB
AMD Ryzen 9 5900 HX	Mobile Processor for bigger computers	512 KB	4MB	16MB
Intel Core i5-6287U	Laptop Processor	128 KB	512 KB	4MB
Intel Core i7-10510U	Laptop Processor	256 KB	1MB	8MB
Intel Core i9-14900K	High-End Processor	256 KB	1MB	8MB

Cache Sizes per Core

Cache Level	Coffee Lake	Skylake Server	Zen 2
Level 1	2x32 KiB	2x32 KiB	2x32 KiB
Level 2	256 KiB	1 MiB	512 KiB
Level 3	1 MiB	1.375 MiB	1 MiB

Cache Latencies (in cycles)

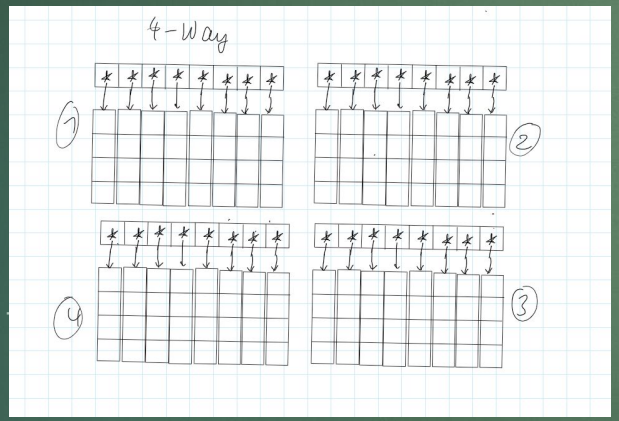
Cache Level	Coffee Lake	Skylake Server	Zen 2
Level 1	4/5	4/5	4-8
Level 2	12	14	12-17
Level 3	42	50-70	40 (average)

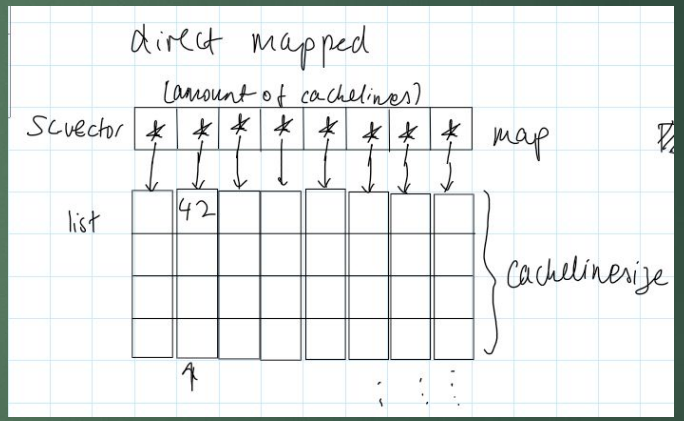
6. Yazeed's work

- Cache module implementation

7. Samhitha's work

- FIrst few iterations of the cache module (later on replaced/overwritten) :





Future outlook

- Optimisation
- Better replacement policy
- Multiple caches (hierarchy)
-